MIT | Academy of Engineering

(An Autonomous Institute Affiliated to Savitribai Phule Pune University)

ASSIGNMENT

AI ML Lab

Practical Assignment 1.2: Reasoning in Knowledge Based Expert System

GitHub Repo : Tushar's GitHub

Field	Details
Name	**Tushar Kaldate**
PRN	202401040191
Roll No.	47
Division	A2
Branch	Computer Engineering
Course	Artificial Intelligence and Machine Learning (AIML)
Academic Year	2026–27
Faculty	Ms. Sunita Barve

1.Title

Programming Language Recommendation System Using Prolog

Aim

To design and develop a **Programming Language Recommendation Expert System using Prolog** by creating a knowledge base of facts and rules that recommends a suitable programming language based on the user's area of interest and skill level.

2.Experimental Objectives

1. To understand the basic concepts of **Prolog programming**.
2. To create a knowledge base using **facts and rules**.
3. To represent knowledge about different programming languages and their application areas.
4. To use Prolog's **logical inference mechanism** for generating recommendations.
5. To recommend suitable programming languages according to the user's **interest and experience level**.
6. To understand the use of **predicates, variables, rules, lists, and queries** in Prolog.
7. To test the expert system using different Prolog queries.
8. To analyze how rule-based systems can be used to solve real-world recommendation problems.

3.Experimental Outcome

After completing this experiment, the student will be able to:

- Understand how an **expert system** can be represented using Prolog.
- Create and manage a Prolog **knowledge base**.
- Differentiate between **facts and rules**.
- Write Prolog queries to retrieve information from the knowledge base.
- Use logical inference to generate recommendations.
- Develop a simple rule-based system for a specialized domain.
- Understand how Prolog can be applied to **decision-making and recommendation systems**.

4.Conceptual Details

i. Expert System

An **expert system** is an artificial intelligence system that attempts to solve problems or provide recommendations using knowledge collected from a particular domain.

An expert system generally consists of:

- **Knowledge Base** – stores facts and rules.
- **Inference Engine** – applies rules to the available facts.
- **User Query** – represents the problem or requirement given to the system.
- **Output** – provides a conclusion, recommendation, or solution.

In this experiment, the specialized domain is **programming language selection**.

For example, if a user is interested in Artificial Intelligence and is a beginner, the system can recommend **Python**.

ii. Prolog

Prolog (Programming in Logic) is a logic programming language commonly used in artificial intelligence and knowledge-based systems.

Instead of describing the exact steps required to solve a problem, Prolog represents:

- **Facts**
- **Rules**
- **Relationships**
- **Queries**

Prolog then uses logical reasoning to determine whether a query is true and to find suitable values for variables.

iii. Facts

Facts represent information that is already known to the system.

For example:

language(python).

language(cpp).

language(java).

These facts tell Prolog that Python, C++, and Java are programming languages.

Other facts can describe their characteristics, such as:

level(python, beginner).

performance(cpp, very_high).

Here, Python is associated with the beginner level, while C++ is associated with very high performance.

iv. Rules

Rules describe relationships or conditions from which new information can be derived.

For example:

recommend(ai, beginner, python).

This represents the knowledge that Python is a suitable recommendation for a beginner interested in Artificial Intelligence.

The system contains similar rules for areas such as:

- Machine Learning
- Data Science
- Web Development
- Android Development
- Game Development
- Competitive Programming
- Systems Programming
- Cloud Computing
- Blockchain

v. Knowledge Base

The knowledge base contains information about different programming languages.

The system considers characteristics such as:

Characteristic	Examples
Programming Language	Python, C++, Java, Rust
Application Domain	AI, Web, Game Development
Skill Level	Beginner, Intermediate, Advanced
Programming Paradigm	Object-Oriented, Functional, Multi-Paradigm
Performance	Medium, High, Very High

This knowledge is used by the inference engine to produce recommendations.

vi. Query

A **query** is a question given to Prolog to obtain information from the knowledge base.

For example:

recommend(ai, beginner, X).

The variable X asks Prolog to find a programming language that satisfies the given conditions.

The system may return:

X = python.

Similarly:

recommend(systems, advanced, X).

can produce:

X = rust.

vii. Inference

Inference is the process through which Prolog uses the available facts and rules to answer a query.

The basic process in this project is:

User Interest + Skill Level → Knowledge Base → Rules → Inference → Recommended Language

For example:

Input:

Interest = Competitive Programming

Level = Advanced

Rule matching:

competitive_programming + advanced → C++

Output:

Recommended Language = C++

viii. Recommendation with Explanation

The system can also provide a reason for the recommendation.

For example:

Interest: AI

Level: Beginner

↓

Recommended Language: Python

↓

Reason: Easy syntax and excellent AI/ML libraries

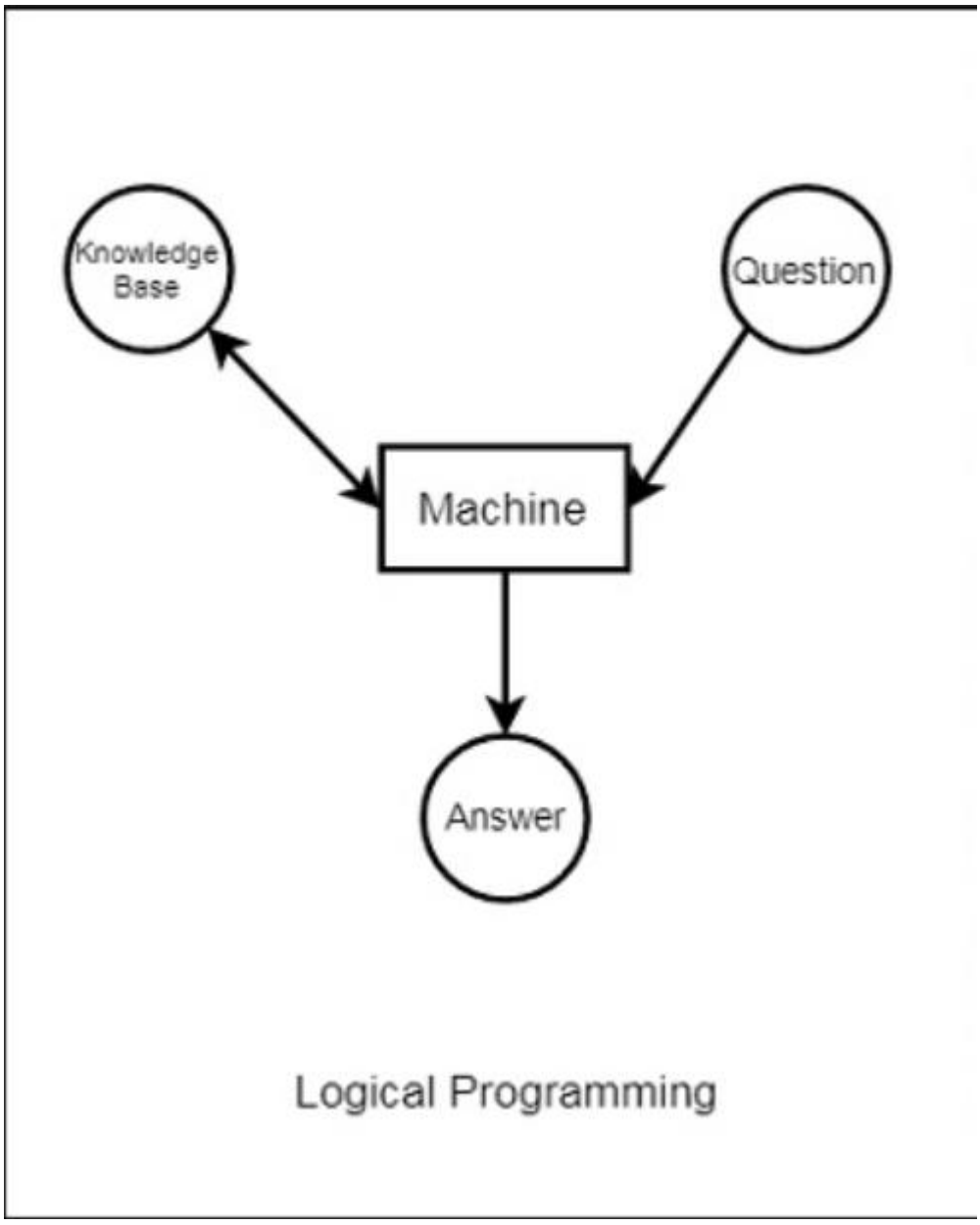
This makes the system more useful than simply returning the name of a programming language.

5. Algorithm

1. Start the program.
2. Create the knowledge base.
3. Store the available programming languages as facts.
4. Store application domains associated with each language.
5. Store the suitable skill levels for each language.
6. Store additional characteristics such as programming paradigm and performance.
7. Define recommendation rules for different interests and skill levels.
8. Accept a query containing the user's interest and skill level.

9. Match the query with the available recommendation rules.
10. Use Prolog's inference mechanism to find a matching programming language.
11. If a matching language is found, display the recommended language.
12. If required, display the reason for the recommendation.
13. Allow the user to execute additional queries.
14. Stop the program.

6. Flowchart



7. CODE :

```
% PROGRAMMING LANGUAGE RECOMMENDATION EXPERT SYSTEM

%  facts

language(python).
language(cpp).
language(java).
language(javascript).
language(typescript).
language(csharp).
language(kotlin).
language(swift).
language(rust).
language(go).
language(r).
language(php).

domain(python, [ai, ml, data_science, automation, web]).
domain(cpp, [competitive_programming, game, systems, embedded]).
domain(java, [enterprise, backend, android]).
domain(javascript, [web, frontend, backend]).
domain(typescript, [web, frontend, backend]).
domain(csharp, [game, desktop, enterprise]).
domain(kotlin, [android, backend, mobile]).
domain(swift, [ios, mobile]).
domain(rust, [systems, embedded, blockchain]).
domain(go, [cloud, backend, distributed]).
domain(r, [data_science, statistics, data_analysis]).
```

```
domain(php, [web, backend]).


level(python, beginner).
level(python, intermediate).
level(cpp, intermediate).
level(cpp, advanced).
level(java, intermediate).
level(javascript, beginner).
level(javascript, intermediate).
level(typescript, intermediate).
level(csharp, beginner).
level(csharp, intermediate).
level(kotlin, beginner).
level(swift, beginner).
level(rust, advanced).
level(go, intermediate).
level(r, beginner).
level(php, beginner).


paradigm(python, multi_paradigm).
paradigm(cpp, multi_paradigm).
paradigm(java, object_oriented).
paradigm(javascript, multi_paradigm).
paradigm(typescript, multi_paradigm).
paradigm(csharp, object_oriented).
paradigm(kotlin, object_oriented).
paradigm(swift, object_oriented).
paradigm(rust, multi_paradigm).
paradigm(go, procedural).
paradigm(r, functional).
paradigm(php, multi_paradigm).
```

```
performance(python, medium).
performance(cpp, very_high).
performance(java, high).
performance(javascript, medium).
performance(typescript, medium).
performance(csharp, high).
performance(kotlin, high).
performance(swift, high).
performance(rust, very_high).
performance(go, high).
performance(r, medium).
performance(php, medium).



%  RULES

% aiml
recommend(ai, beginner, python).
recommend(ai, intermediate, python).
recommend(ml, beginner, python).
recommend(ml, intermediate, python).

% ds
recommend(data_science, beginner, python).
recommend(data_science, intermediate, r).

% Web Dev
recommend(web, beginner, javascript).
recommend(web, intermediate, typescript).
recommend(frontend, beginner, javascript).
```

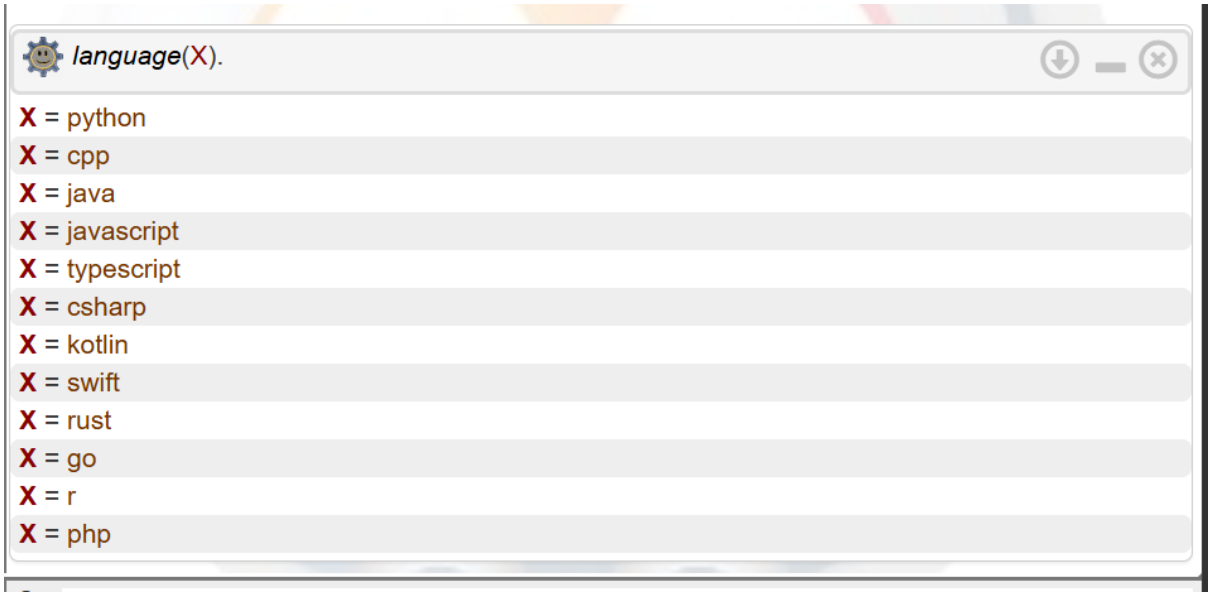
```
recommend(frontend, intermediate, typescript).
recommend(backend, intermediate, java).

% Mobile Dev
recommend(android, beginner, kotlin).
recommend(android, intermediate, kotlin).
recommend(ios, beginner, swift).
recommend(mobile, intermediate, swift).

% Game Dev
recommend(game, beginner, csharp).
recommend(game, intermediate, cpp).

% CP
recommend(competitive_programming, beginner, cpp).
recommend(competitive_programming, intermediate, cpp).
recommend(competitive_programming, advanced, cpp).

% Systems and Embedded
recommend(systems, intermediate, cpp).
recommend(systems, advanced, rust).
recommend(embedded, intermediate, cpp).
recommend(embedded, advanced, rust).

% Cloud
recommend(cloud, intermediate, go).
recommend(distributed, advanced, go).

% Blockchain
recommend(blockchain, advanced, rust).
```

```
%  RULES

reason(python, 'Easy syntax and excellent AI/ML libraries').
reason(cpp, 'High performance and powerful STL').
reason(java, 'Reliable language for enterprise applications').
reason(javascript, 'Essential language for modern web development').
reason(typescript, 'Adds static typing to JavaScript').
reason(csharp, 'Popular choice with Unity game development').
reason(kotlin, 'Modern language officially supported for Android').
reason(swift, 'Designed for Apple and iOS development').
reason(rust, 'Memory safety with very high performance').
reason(go, 'Simple, fast and excellent for cloud systems').
reason(r, 'Strong statistical and data analysis features').
reason(php, 'Simple and widely used for server-side web development').

% QUERY PREDICATES

% Find languages suitable for a domain
language_for_domain(Domain, Language) :-
    domain(Language, Domains),
    member(Domain, Domains).

% Find languages suitable for a skill level
language_for_level(Level, Language) :-
    level(Language, Level).

% Find language with required performance
language_with_performance(Level, Language) :-
    performance(Language, Level).

% Get recommendation with explanation
```

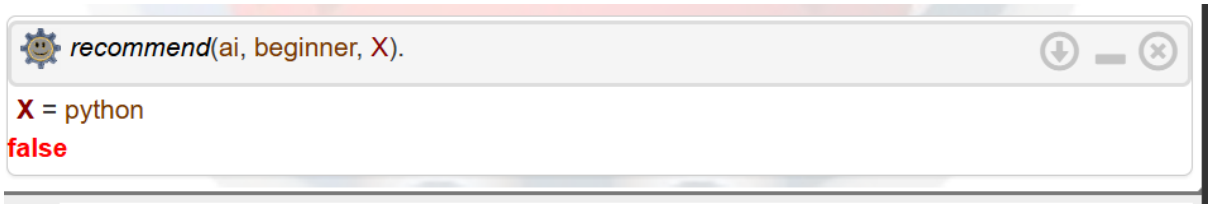
```
recommend_with_reason(Interest, Level, Language, Reason) :-
    recommend(Interest, Level, Language),
    reason(Language, Reason).

% Get complete information about a language
language_details(Language, Domains, Level, Paradigm, Performance) :-
    domain(Language, Domains),
    level(Language, Level),
    paradigm(Language, Paradigm),
    performance(Language, Performance).
```

8. OUTPUT :

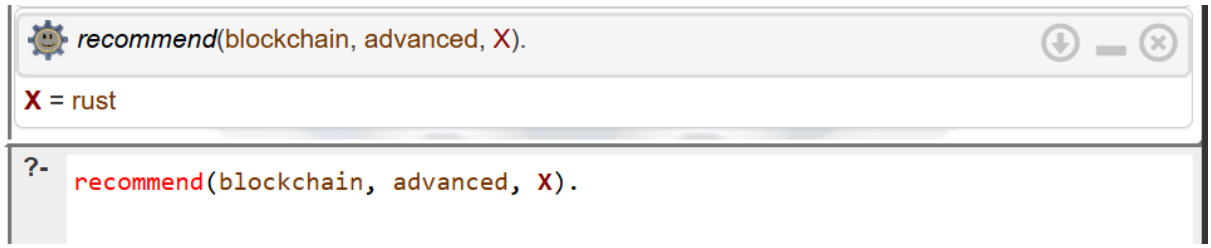
1.Language(X) :



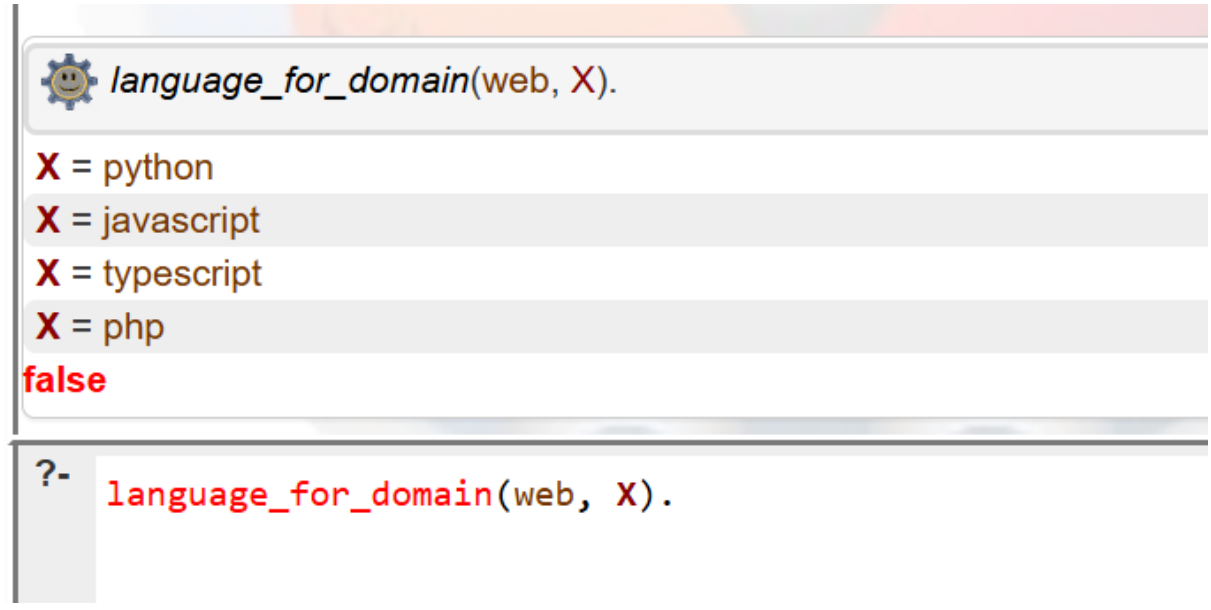
2. recommend(ai, beginner, X).



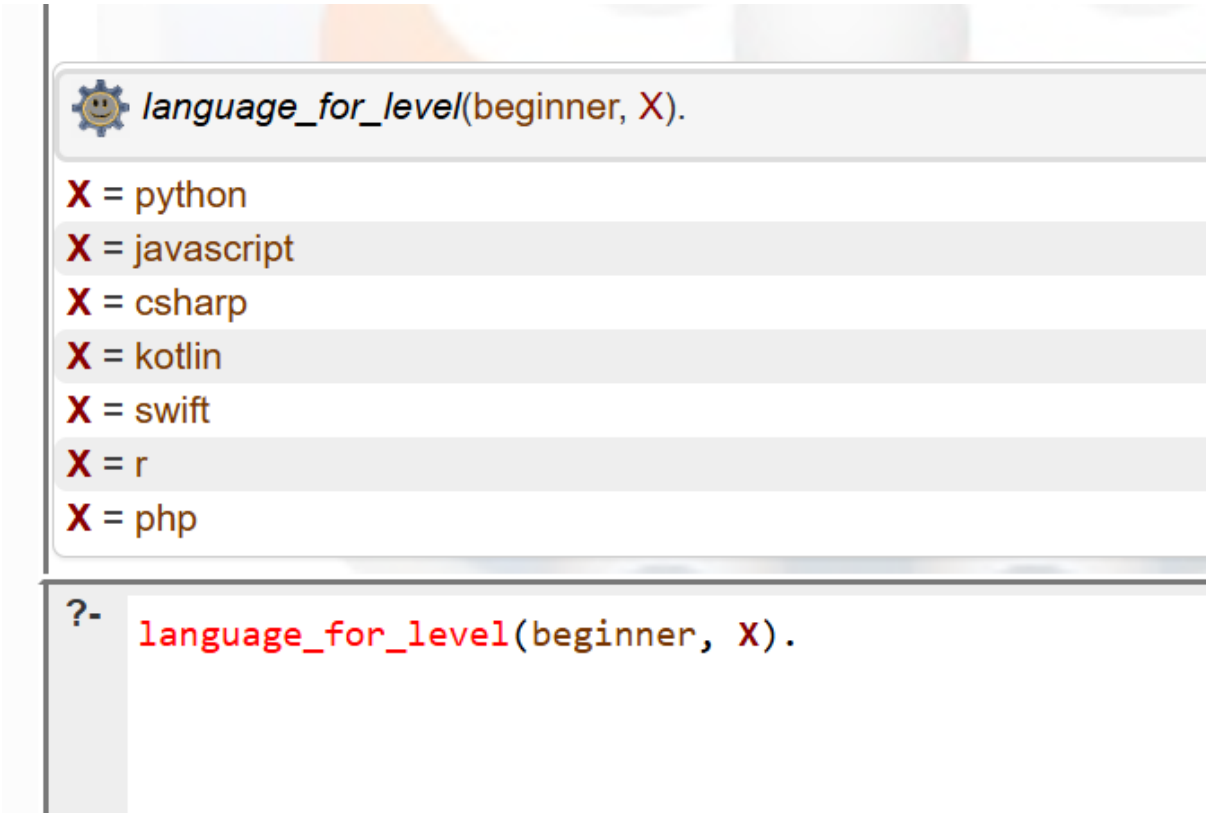
3. recommend(blockchain, advanced, X).



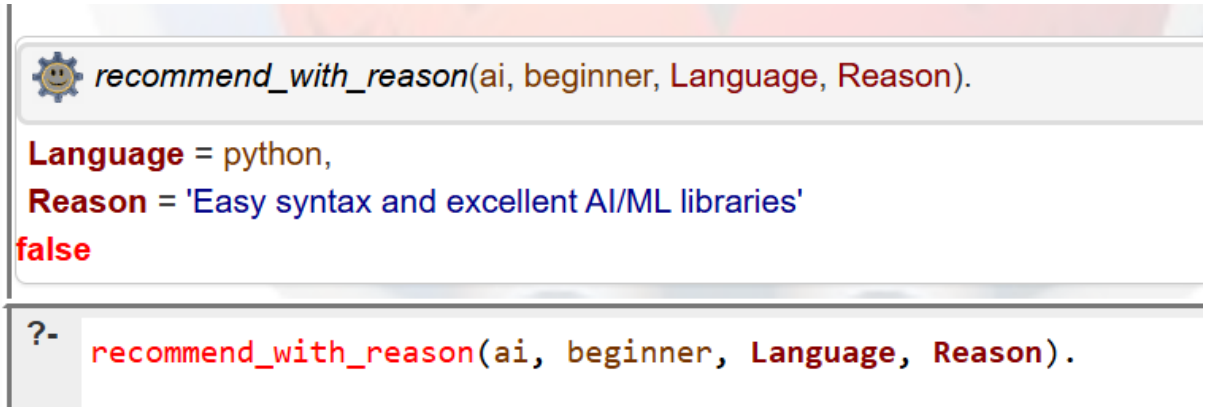
4. language_for_domain(web, X).



5. language_for_level(beginner, X).



6. recommend_with_reason(ai, beginner, Language, Reason).



7. language_details(python, Domains, Level, Paradigm, Performance).

language_details(python, Domains, Level, Paradigm, Performance).

Domains = [ai, ml, data_science, automation, web],
Level = beginner,
Paradigm = multi_paradigm,
Performance = medium
Domains = [ai, ml, data_science, automation, web],
Level = intermediate,
Paradigm = multi_paradigm,
Performance = medium

```
?- language_details(python, Domains, Level, Paradigm, Performance).
```

9. Results Analysis

The Programming Language Recommendation System was successfully implemented using Prolog. The knowledge base contained information about multiple programming languages, their application domains, skill levels, programming paradigms, and performance characteristics.

Different queries were tested to verify the working of the expert system.

For example, when the query was:

recommend(ai, beginner, X).

the system returned:

X = python.

This indicates that Python was recommended for a beginner interested in Artificial Intelligence.

Similarly, for:

recommend(competitive_programming, advanced, X).

the system returned:

X = cpp.

For an advanced user interested in systems programming:

recommend(systems, advanced, X).

the system returned:

X = rust.

The system was also able to provide explanations for recommendations. For example, Python was recommended for AI because of its simple syntax and extensive AI/ML libraries, while Rust was recommended for systems programming because of its performance and memory-safety features.

The experiment demonstrates that Prolog can effectively represent domain-specific knowledge using **facts and rules** and can use logical inference to derive appropriate conclusions from user queries.

10.Conclusion

The **Programming Language Recommendation System** was successfully developed using Prolog. The experiment demonstrated how an expert system can be created by combining a knowledge base of facts with logical rules.

The system can recommend programming languages based on factors such as the user's **area of interest and skill level**. Prolog's inference mechanism automatically matches the user's query with the appropriate rules and generates the required recommendation.

Through this experiment, the concepts of **facts, rules, predicates, variables, queries, knowledge representation, and logical inference** were understood and applied practically. The project also demonstrates how Prolog can be used to build simple AI-based decision-making and recommendation systems.